

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

УДК 004.75

Отчет об исследовательском проекте на тему:
Системы балансировки нагрузки в гибридных распределенных системах

Выполнил студент:

группы #БПМИ236, 3 курса

Разухин Александр Сергеевич

Принял руководитель проекта:

Курочкин Илья Ильич
к.т.н., зав.лаб. Ц-1
ИППИ РАН

Москва 2026

Содержание

Аннотация	4
1 Введение	5
1.1 Описание предметной области	5
1.2 Актуальность и значимость	5
1.3 Цели и задачи	6
1.4 Состав результатов проекта	6
2 Литературный обзор	7
2.1 Обзор предметной области	7
2.2 Обзор существующих методов	7
2.3 Особенности BOINC	8
2.4 Сравнение подходов	8
2.5 Выводы из обзора	9
3 Моделирование системы	10
3.1 Выбор способа моделирования	10
3.2 Топология системы	10
4 Подходы к тестированию системы и метрики качества	12
4.1 Эвристики распределения задач	12
4.2 Метрики оценки производительности	12
4.3 Процесс тестирования	13
5 Разработка алгоритма балансировки	14
5.1 Планировщик BOINC по умолчанию	14
5.2 Требования к кастомному алгоритму	16
5.3 Разработка планировщика задач в гетерогенной системе	16
5.4 Подбор оптимальных параметров для алгоритма	19
5.5 Сопоставление подходов в контексте BOINC	20
5.6 Ограничения разработанного подхода	20
6 Результаты тестирования и сравнение	21
6.1 Стабильная среда	21
6.2 Нестабильная среда: $f_{\text{off}} \approx 32\%$	22

6.3	Нестабильная среда: $f_{\text{off}} = 20\%$	24
6.4	Нестабильная среда: $f_{\text{off}} = 10\%$	25
6.5	Нестабильная среда: $f_{\text{off}} = 50\%$	25
6.6	Нестабильная среда: $f_{\text{off}} \approx 68\%$	26
6.7	Сводное сравнение сценариев	26
6.8	Интерпретация результатов	27
7	Заключение	28
7.1	Подтверждённые результаты	28
7.2	Ограничения и компромиссы	28
	Список литературы	30

Аннотация

В работе исследуется выдача заданий в гетерогенной среде BOINC с помощью локального ранжирования задач при запросе клиента. Реализована эвристика *hybrid*, учитывающая оценку длительности задачи, условную мощность узла и статистику доступности BOINC. Создан тестовый контейнерный стенд из шести клиентов четырёх различных ресурсных профилей. В каждом сценарии выполнено сравнение штатного планировщика и шести экспериментальных политик на наборе синтетических заданий. *Hybrid* повышает значения коэффициента Джини по числу завершённых задач относительно *baseline* во всех шести основных сравнениях, но сокращает *makespan* только в одном из них. Установлен компромисс между равномерностью числа задач и временем обработки их фиксированного набора, описаны условия применимости результатов.

Ключевые слова

Распределенные системы, балансировка нагрузки

1 Введение

1.1 Описание предметной области

Предметная область проекта охватывает распределенные системы с гетерогенной инфраструктурой с основными компонентами:

- BOINC (Berkeley Open Infrastructure for Network Computing) — платформа для добровольных вычислений, объединяющая ресурсы разнородных устройств (серверов, кластеров, персональных компьютеров) в единую сеть
- Кластерные ресурсы — группа вычислительных узлов под общим управлением
- Гетерогенные ресурсы — устройства с различными характеристиками: вычислительной мощностью, объемом памяти, доступностью и другими
- Балансировщик нагрузки — компонент, отвечающий за распределение вычислительных задач между узлами с учетом их возможностей и текущих состояний

Также предметная область имеет следующие особенности:

- Отсутствие централизованного контроля за узлами системы
- Разнородность ресурсов каждого из узлов
- Различные режимы работы узлов (не все устройства доступны на протяжении 100% времени)
- Различный объем задач, время выполнения которых может варьироваться от нескольких минут до нескольких дней
- Необходимость минимизации накладных расходов для балансировщика

1.2 Актуальность и значимость

Актуальность данного проекта обусловлена множеством существующих на данный момент проблем, таких как:

- 1 Современные вычислительные задачи такие, как анализ больших данных, биоинформатика, требуют огромного количества ресурсов, что способствует росту числа гетерогенных распределенных систем

- 2 BOINC учитывает производительность, доступность и совместимость узлов, однако локальная выдача задач в ответ на запрос клиента не обеспечивает заданного глобального распределения нагрузки
- 3 Необходимость развития динамических алгоритмов балансировки в противовес существующим решениям с их статическими правилами
- 4 Исследование компромисса между скоростью выполнения набора заданий и распределением работы между разнородными клиентами

1.3 Цели и задачи

Целью данного проекта является разработка системы выдачи заданий для распределенной системы на платформе BOINC. Для ее достижения необходимо было выполнить следующие задачи:

- 1 Обзор систем балансировки нагрузки для кластеров и распределенных систем
- 2 Выбор системы моделирования и адаптация модели под специфику гибридной распределенной системы
- 3 Реализация/модификация системы балансировки нагрузки гибридной распределенной системы
- 4 Тестирование системы балансировки на модели гибридной распределенной системы
- 5 Анализ полученных результатов

1.4 Состав результатов проекта

- 1 Подробное описание методики моделирования гибридной распределенной системы
- 2 Подробное описание системы балансировки нагрузки гибридной распределенной системы
- 3 Анализ результатов моделирования системы
- 4 Подготовка подробного отчета о проделанной работе и проведенном тестировании

2 Литературный обзор

2.1 Обзор предметной области

Балансировка нагрузки отвечает за распределение работы между несколькими вычислительными ресурсами. Необходимо отличать маршрутизацию сетевых запросов от планирования длительных вычислительных заданий: для второго критически важны оценка трудоёмкости, ограничения ресурсов, сроки получения результата и доступность клиента-исполнителя. Поэтому классические методы серверной и облачной балансировки [3, 5] требуют адаптации к системе BOINC.

Статические методы используют заранее заданные правила, например циклический порядок или случайный выбор. Динамические методы используют сведения о текущем состоянии ресурсов; отдельные варианты допускают перенос работы, но миграция не является обязательным свойством динамического планирования [1].

BOINC разделяет серверную выдачу заданий от клиентского запрашивания работы и локального планирования. Эти механизмы постоянно взаимодействуют между собой, поэтому результат серверной эвристики зависит также от частоты запросов и настроек клиента [2].

2.2 Обзор существующих методов

Рассмотрим методы балансировки нагрузки, применимые к распределённым вычислительным системам с акцентом на гибридные конфигурации (платформы типа BOINC + стабильные кластерные сегменты). Среди критериев для них можно выделить: простота мониторинга, устойчивость к churn (неконтролируемое подключение / отключение узлов), гетерогенность ресурсов, величина накладных расходов, масштабируемость. Всего можно выделить несколько групп подходов к балансировке нагрузки:

- Классические статические методы (Round Robin, Random Strategy) — просты в мониторинге, однако не учитывают размер и время выполнения задач
- Классические динамические методы (Least Connections) — используют число соединений как приближение нагрузки и не учитывают вычислительную трудоёмкость заданий
- Адаптивные методы — балансировка происходит на основе метрик, вычисляемых в реальном времени; требуют постоянных вычислений
- Предиктивные ML-подходы — методы с обучаемой оценкой нагрузки; их качество зависит от обучающих данных и соответствия условий применения [6]

- Существующие решения для BOINC (CluBORun) — интеграция кластерных ресурсов вычисления [7]

Большинство классических и оптимизационных алгоритмов работают при допущении, что узлы доступны и их характеристики относительно стабильны. В средах типа BOINC это допущение неприменимо: изменчивость системы и разнородность устройств приводит к частому перераспределению задач между узлами. Если в гомогенных системах перенос задач - относительно стандартный процесс, в гибридных системах такой подход может привести к серьёзному росту накладных расходов из-за неоднородности устройств, поэтому в подобных системах правильным подходом считается исключение или сведение к минимуму подобных операций. Также нужно отметить, что для гибридных систем важно сохранять баланс между количеством собираемых метрик и их полезности для планировщика, что труднодостижимо при подходах с глобальным мониторингом узлов. [5]

2.3 Особенности BOINC

Платформа BOINC использует модель client-initiated: вычислительный узел сам запрашивает работу через RPC. Scheduler сканирует массив feeder, оценивает совместимость задач с хостом и ранжирует кандидатов по score. Назначенная задача не мигрирует на другой host до таймаута или повторной репликации. Поле *rsc_fpop_est* задаёт оценку сложности, которая может расходиться с реальным временем выполнения - это важный источник ошибок для любой adaptive-политики. Решение CluBoRun [4] интегрирует кластерные ресурсы в BOINC-проект, но не решает глобальную балансировку по классам устройств с учетом их мощности.

2.4 Сравнение подходов

Поймем, насколько применимы указанные подходы к гибридным системам и выделим их недочеты:

Таблица 2.1: Сравнение групп подходов к балансировке нагрузки в гибридных системах

Группа методов	Тип	Требования мониторинга	Цель подхода	Минусы
Классические статические	Простые	Низкие	Простое планирование	Не учитывают размер задач
Классические динамические	Реактивные	Низкие/средние	Сократить число одновременных задач на узле	Число соединений не отражает трудоёмкость
Адаптивные методы	С обратной связью	Средние или высокие	Снижение задержки и выравнивание нагрузки	Расходы на сбор состояния
Предиктивные подходы	ML Обучаемые	Высокие	Минимизация целевой функции ошибки	Требуют данных и обучения
CluBORun	Интеграция кластера / устройств	Низкие/средние	Использование idle ресурсов, учёт доступности	Иная задача: интеграция ресурсов

2.5 Выводы из обзора

- На данный момент существует широкий набор методов балансировки, однако немногие работы систематически исследуют их поведение в гибридных системах
- Основные пробелы: недостаточная оценка влияния churn, упрощённое моделирование сетевых затрат при распределении задач, отсутствие оценки накладных расходов мониторинга и планировщика
- Нет ни одной работы, в которой бы честно рассматривалась балансировка нагрузки в гибридной системе на базе BOINC

Данная работа направлена на устранение выявленных ограничений за счёт оптимизации накладных расходов планировщика, более точного учёта динамической доступности узлов и повышения общей производительности системы при рациональном использовании вычислительных ресурсов.

3 Моделирование системы

3.1 Выбор способа моделирования

Для проверки политик балансировки выбран локальный стенд на базе Docker Compose и реального планировщика BOINC 8.3.0, а не discrete-event симулятор или GPU-кластер. Такой выбор обусловлен следующими соображениями:

- Сохраняется полный RPC-цикл BOINC (feeder, feasibility checks, replication, обновление БД)
- Результаты воспроизводимы через скрипты и конфигурационные файлы из артефактов каждого запуска
- Гетерогенность задаётся через Docker cgroup и BOINC preferences без покупки реальной группы устройств с реальным GPU
- Синтетическая нагрузка задаётся встроенным приложением для генерации задач с параметром `—cpu_time`, отвечающим за нагрузку на процессор

Основной эксперимент - CPU-only: отсутствие физических устройств с собственным GPU не позволяет построить корректную модель, а варианты с разворачиванием виртуальных машин лишь имитируют наличие GPU. Также использование GPU смещает фокус с основной идеи BOINC - volunteer CPU computing.

3.2 Топология системы

Тестовый стенд состоит из 6 хостов-устройств с различными характеристиками:

- 2 кластерных элемента (4 CPU, 4 GiB RAM)
- 1 устройство ПК (2 CPU, 2 GiB RAM)
- 1 маломощный узел (1 CPU, 1 GiB RAM)
- 2 элемента - мобильных устройств (0.5 CPU, 0.5 GiB RAM)

Каждое устройство представляет собой независимый вычислительный хост; с точки зрения технической реализации каждый хост - это отдельный Docker контейнер, который конфигурационно настроен под конкретное число CPU. Данный выбор хостов с их настройками был обусловлен следующими требованиями к тестовому стенду:

- Стенд обязан содержать разнородные с точки зрения мощности устройства для моделирования гибридной среды
- Стенд обязан содержать как высокопроизводительные элементы (кластеры), так и низкопроизводительные (мобильные устройства)
- Стенд обязан иметь несколько одинаковых устройств для проверки балансировки нагрузки между однородными хостами (распределение задач между одинаковыми с точки зрения мощности клиентами)

Таким образом и была сформирована модель системы, используемая при тестировании.

Протокол churn Отказы моделируются через `docker compose pause/unpause` на всех шести клиентах с рассинхронизацией старта (разница в начале циклов отказа конфигурируется через `CHURN_STAGGER_SECONDS`). Каждый хост проходит несколько циклов `online` → `pause` → `unpause`. Доля `offline` в одном цикле:

$$f_{\text{off}} = \frac{T_{\text{offline}}}{T_{\text{online}} + T_{\text{offline}}}.$$

Например, при 25/12 получаем $f_{\text{off}} \approx 32\%$, при 40/10 - 20%, при 12/25 - 68%. Все события (включение / отключение хоста) логируются для последующего вычисления метрики Load CV.

Допущения тестового стенда Все шесть клиентов используют одну физическую машину и упрощённую сеть. CPU-квоты Docker и настройки BOINC моделируют ограничения ресурсов, но не различные микроархитектуры процессоров. Ограничения `max_wus_to_send = 1` и `max_wus_in_progress = 2` ограничивают захват очереди задач более быстрыми хостами. Эти допущения не отменяют А/В-сравнение политик на одном стенде, однако не позволяют напрямую экстраполировать абсолютные значения метрик на production систему.

4 Подходы к тестированию системы и метрики качества

4.1 Эвристики распределения задач

В качестве контрольных эвристик в проекте реализованы:

- LPT (Longest Processing Time first) - приоритет длинным задачам; проверяется гипотеза о сокращении общего времени завершения набора
- SJF (Shortest Job First) - приоритет коротким задачам; проверяется гипотеза о снижении среднего времени отклика, при этом крупные задачи могут простаивать в очереди дольше

Slurm и Torque работают в другой модели управления ресурсами и не включены в экспериментальное сравнение: их полноценная интеграция потребовала бы другого стенда, а также они неприменимы к гетерогенной структуре BOINC. На стенде сравниваются baseline, hybrid и локальные реализации эвристик random, LPT, SJF, Round Robin и Weighted Least Loaded.

4.2 Метрики оценки производительности

Выбор метрик обусловлен необходимостью учета как общих показателей производительности, так и особенностей гетерогенной вычислительной среды.

- Пропускная способность (Throughput) - число выполненных задач за единицу времени. Формула: $T = \frac{N_{\text{completed}}}{\Delta t}$, где Δt - время от начала выполнения задач до его окончания
- Загруженность узлов - для узла j : $u_j = \frac{\text{cpu_time}_j}{\text{online_seconds}_j \cdot \text{nrcpus}_j}$. Сводная метрика - Load CV $= \sigma(u_j)/\bar{u}$ - коэффициент вариации CPU-нагрузки; может расходиться с Jain
- Общая продолжительность выполнения задач - $Makespan = \max_i t_{\text{finish}}^{(i)} - \min_i t_{\text{create}}^{(i)}$
- Равномерность числа завершённых задач (Jain) - $J = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2}$, где x_i - число завершённых задач на хосте i ; в расчёт попадают только хосты с $x_i \geq 1$
- Response - время от создания задачи до получения результата сервером; p95 - 95-й перцентиль

Такой набор метрик позволяет оценить и скорость системы (throughput, makespan, response), и качество распределения (Jain, Load CV). Отдельно анализируется согласованность типов задач с классами устройств (таблицы short/medium/long по хостам).

4.3 Процесс тестирования

Система тестировалась в шести сценариях режима: одна стабильная среда и пять вариантов churn с разными $(T_{\text{online}}, T_{\text{offline}})$. Метрики автоматически вычисляются скриптом *analyze_metrics.py* и сохраняются в файл *analysis/metrics.json*. Параметры прогона - в файл *manifest.json*.

В качестве источника задач использовалось приложение *uppercase* с параметром *-cpu_time*. Нагрузка: 36 задач - 18 short (5 s), 12 medium (30 s), 6 long (120 s) - достаточный набор для ограниченного во времени эксперимента. Основная цель кастомного алгоритма - оптимизировать распределение задач по хостам, направив long/medium на более мощные узлы, а short - на менее мощные, найдя компромисс с остальными метриками.

Таблица 4.1: Сценарии тестирования

Churn	Online (s)	Offline (s)	f_{off}
stable	—	—	0
heavy	45	5	10%
heavy	40	10	20%
heavy	25	12	32%
heavy	30	30	50%
heavy	12	25	68%

5 Разработка алгоритма балансировки

Прежде чем описывать разработанный алгоритм, необходимо зафиксировать, как именно работает штатный планировщик в BOINC, поскольку именно он выступает baseline в экспериментах и служит границей, ниже которой по метрикам опускаться неприемлемо.

5.1 Планировщик BOINC по умолчанию

BOINC использует pull-модель распределения задач: вычислительный узел самостоятельно инициирует RPC-запрос к планировщику и сообщает, сколько работы он готов принять (в секундах CPU, в числе задач или иначе). Планировщик не выбирает хост для задачи - он лишь отвечает на запрос данного клиента в данный момент. Назначенная задача не мигрирует на другой узел по достижении таймаута, при ошибке или при создании новой реплики. Это принципиально отличает BOINC от централизованных кластерных планировщиков (Slurm, Torque), где планировщик видит всю очередь задач и все узлы одновременно, и именно эти недостатки должны быть устранены или сведены к минимуму в рамках текущего проекта.

Очередь задач, доступная планировщику, - это не вся база данных, а кэш в памяти, который заполняет отдельный процесс под названием *feeder*. Он читает из БД информацию о задачах, готовых к отправке, и помещает их в массив фиксированного размера (по умолчанию до 100 слотов, параметр *shmem_work_items* задаётся конфигурационно). Каждый элемент содержит метаданные задачи (оценку сложности *rsc_fpop_est*, требования к памяти и диску), приоритет реплики *res_priority* и счётчик отказов *infeasible_count*. Планировщик при каждом запросе клиента сканирует этот массив; порядок обхода начинается со случайного смещения, чтобы не застревать на одних и тех же задачах.

Для выбора задач текущему хосту в выбранной конфигурации вызывается функция *send_work_score()*, логика которой сводится к двум шагам:

- 1 Сканирование и ранжирование - для каждого слота в shared memory подбирается совместимая версия приложения под платформу хоста, для каждой допустимой пары (*workunit*, *host*) вычисляется *score*, после чего все кандидаты сортируются по *score* по убыванию
- 2 Отправка - планировщик последовательно пытается выдать задачи из отсортированного списка, пока не исчерпан лимит RPC *max_wus_to_send*, не превышены квоты *max_jobs_in_progress* и пока хост ещё запрашивает работу

Отдельный проход выполняется для каждого типа процессора (GPU-типы, затем CPU).

Перед выдачей проверяется допустимость задания для хоста:

- Достаточно ли оперативной памяти и дискового пространства?
- Приложение входит в список, выбранный пользователем (хостом) в project preferences?
- Хост успеет завершить задачу до глобального дедлайна по ней?

Последняя проверка - ключевая для гетерогенной среды. Оценка длительности выполнения задачи:

$$\hat{t}_{ij} = \frac{\text{rsc_fpops_est}_i}{\text{projected_flops}_j \cdot \text{available_frac}_j},$$

Где *available_frac* - доля времени, когда клиент статистически доступен для BOINC. К этой оценке добавляется *estimated_delay* - время, которое хост уже занят текущими задачами. Если $\hat{t}_{ij} + \text{estimated_delay} > \text{delay_bound}$, задача для данного хоста в score не попадает. Следствие: мощные и стабильные узлы забирают длинные задачи; слабые и малодоступные - только короткие или остаются без задач.

Если быстрые проверки пройдены, выполняется *slow_check()* с запросами в БД: нет ли уже результата этой задачи на данном хосте. Только после этого задача добавляется в ответ клиенту.

Оценка каждого хоста считается исходя из нескольких эвристик:

- Reliability - при требовании надёжности проверяется признак *reliable* соответствующей версии приложения на хосте
- LTT - для *high - priorityworkunits* в режиме *batchacceleration* задачи идут только на LTT-хосты
- Locality - бонус, если входные данные задачи уже есть на хосте
- *res_priority* - приоритет реплики из БД добавляется напрямую к оценке
- App preferences - учёт пожеланий пользователя

Из этих факторов и складывается итоговый score.

Что baseline оптимизирует и чего не делает. Штатный планировщик оптимизирует локальную выгоду для текущего RPC клиента. При этом он не учитывает глобальную загрузку хостов, не различает классы устройств - только доступность конкретного хоста и его характеристики. Таким образом, возникает проблема: мощные хосты забирают любые задачи, которые им предлагает планировщик, а более слабым узлам достаются не самые простые с точки зрения требуемой вычислительной мощности задачи, из-за чего те вынуждены простаивать.

5.2 Требования к кастомному алгоритму

Исходя из обзора литературы, ограничений штатного планировщика BOINC и целей данной работы, к разрабатываемому алгоритму были сформулированы следующие требования:

- 1 Минимальная инвазивность - сохранить весь контракт штатного scheduler: feeder, feasibility checks, replication, locking shared memory, обновление БД. Меняется только функция ранжирования допустимых кандидатов
- 2 *Size — aware dispatch* - ключевая идея: эвристическое сопоставление оценки размера задачи с ресурсным профилем узла
- 3 *Churn — awareness* - учёт динамической доступности узла, чтобы не отправлять тяжёлые задачи на малодоступные хосты
- 4 Отсутствие миграции - не вводить перенос задач между хостами, поскольку такая возможность потребовала бы отдельного механизма переноса состояния приложения
- 5 Минимизация накладных расходов - сканирование кэша и сортировка кандидатов без введения глобального мониторинга; при M слотах и K кандидатах сложность ранжирования составляет $O(M + K \log K)$
- 6 Сравнимость - помимо целевой политики реализовать контрольные алгоритмы (random, LPT, SJF, Round Robin, Weighted Least Loaded) для сравнения качества распределения и накладных расходов

5.3 Разработка планировщика задач в гетерогенной системе

Основная реализация находится в *sched_custom_load_balancer.cpp*, переключение режима выполняется в *sched_send.cpp*. При включённом флаге вместо встроенной вызывается функция *send_work_custom()*, которая повторяет структуру оригинальной *send_work_score()*:

- Сканирование *ssp* — $\rightarrow$ *wu_results*[] со случайным offset
- Подбор *app_version* и *feasibility* через те же методы *get_score()*, *wu_is_infeasible_fast()*, *slow_check()*
- Сортировка кандидатов по score и отправка top- N

Отличие - в методе *custom_score()*, которая заменяет штатный *JOB::get_score()* для вычисления итоговой оценки хоста.

Поскольку Docker-клиенты часто сообщают одинаковый p_flops , для hybrid введён коэффициент $host_speed_factor = v_j$ по числу CPU узла. Спрогнозированное время в этом случае: $\hat{t}_{ij} = estimate_duration(i, j)/v_j$.

Основной идеей было взять за основу оценку baseline-алгоритма и дополнить ее членами, явно учитывающими размер задачи, доступность хоста и "соответствие" мощности хоста размеру задачи. Целевая политика - hybrid, остальные служат контрольными для изоляции отдельных гипотез.

Контрольные политики Все контрольные политики используют защищённый *baseline score* ($100 \times s^{BOINC}$), чтобы штатные факторы (*reliability*, *locality*) сохраняли влияние, а экспериментальный член мог их сдвинуть:

Таблица 5.1: Реализованные политики

Политика	Суть ранжирования
baseline	штатный <code>send_work_score()</code>
random	baseline + детерминированный псевдослучайный шум от <i>result_id</i> и <i>host_id</i>
lpt	baseline + $\log(1 + \hat{t}_{ij})$ - длинные задачи первыми
sjf	baseline - $\log(1 + \hat{t}_{ij})$ - короткие задачи первыми
round_robin	baseline + циклический score по <i>result_id</i> mod 6 и <i>host_id</i>
weighted_least_loaded	baseline + $\log(1 + v_j \cdot a_j / (L_j + 1))$, где L_j - число задач in-progress на хосте
hybrid	кастомный score

Здесь v_j - *host_speed_factor* (число CPU у хоста), a_j - *cpu_available_frac*.

Кастомная политика Для пары (workunit i , host j) вычисляется составной score. Сначала определяются вспомогательные величины:

$$a_j = \text{clamp}(\text{cpu_available_frac}_j, 0.15, 1.0),$$

$$\tilde{t}_{ij} = \hat{t}_{ij}/a_j,$$

$$\rho_{ij} = \text{clamp}(\tilde{t}_{ij}/T_{\text{target}}, 0.01, 100),$$

где $T_{\text{target}} = 30$ секунд - целевое время выполнения (параметр *custom_lb_target_runtime*, близко к длительности medium-задач). Далее:

- $size_affinity_{ij} = -|\log \rho_{ij}| \cdot a_j$ - штраф за несоответствие размера задачи целевому runtime
- $deadline_press_i = T_{\text{target}}/\text{delay_bound}_i$ - бонус срочным задачам

- $runtime_penalty_{ij} = \log(1 + \tilde{t}_{ij}/T_{\text{target}})$ - штраф за длинные задачи на текущем хосте
- $host_fit_{ij} = (\log(1 + v_j) - \log(1 + \tilde{t}_{ij})) \cdot a_j^2$ - бонус за соответствие мощности хоста длительности задачи

Дополнительный штраф $P_{ij} = \text{slow_host_long_penalty}$ задан явно:

$$P_{ij} = \begin{cases} 1.5 \frac{\hat{t}_{ij}}{T_{\text{target}}} \frac{2 - v_j}{a_j}, & \hat{t}_{ij} > T_{\text{target}}/2, v_j < 2, \\ 0.5 \left(\frac{\hat{t}_{ij}}{T_{\text{target}}} - 1 \right) \frac{4 - v_j}{a_j}, & \hat{t}_{ij} > T_{\text{target}}, v_j < 4, \\ 0, & \text{иначе.} \end{cases}$$

Baseline score масштабируется с учётом доступности хоста, что ослабляет влияние *reliability / locality* на нестабильных хостах и усиливает на стабильных - важно для сценариев с отказами узлов:

$$s_{ij}^{\text{base}} = (10 + 10 \cdot a_j) \cdot s_{ij}^{\text{BOINC}},$$

Итоговый score выглядит так:

$$\begin{aligned} \text{score}_{ij} = & s_{ij}^{\text{base}} + w_{\text{size}} \cdot \text{size_affinity}_{ij} + w_{\text{dl}} \cdot \text{deadline_press}_i \\ & - w_{\text{rt}} \cdot \text{runtime_penalty}_{ij} + 0.05 \cdot w_{\text{size}} \cdot \text{host_fit}_{ij} - w_{\text{rt}} \cdot \text{slow_host_long_penalty}_{ij}, \end{aligned}$$

Веса задаются в config.xml: $w_{\text{size}} = 2$, $w_{\text{dl}} = 5$, $w_{\text{rt}} = 0.55$.

Интерпретация элементов score

- *size_affinity* предпочитает эффективные задачи, по времени выполнения близкие к T_{target} . При фиксированном ρ модуль штрафа уменьшается, но сама ρ также зависит от доступности.
- В текущем RPC v_j постоянно: слагаемое $\log(1 + v_j)$ внутри *host_fit* не меняет порядок задач. Переменная часть *host_fit* предпочитает меньшую длительность; соответствие крупных задач мощным узлам задаётся совокупностью членов, прежде всего *size_affinity* и P_{ij} .
- *runtime_penalty* и *slow_host_long_penalty* не дают накапливать длинные задачи на мало-мощные узлы
- *deadline_press* предотвращает долгое ожидание взятия в работу для задач с коротким таймаутом

- Множитель $(10 + 10 \cdot a_j)$ вместо 100 у контрольных политик - компромисс: baseline-факторы BOINC сохраняют значимость, но кастомные слагаемые также могут играть важную роль при формировании финальной оценки хоста

Параметры конфигурации Переключение политик и весов выполняется через файл config.xml без пересборки:

- *custom_load_balancer* - 0 (baseline) / 1 (custom)
- *custom_lb_policy* - имя политики
- *custom_lb_target_runtime* - целевая длительность
- *custom_lb_size_weight* - вес соответствия размера
- *custom_lb_deadline_weight* - вес срока возврата
- *custom_lb_runtime_weight* - вес штрафов длительности
- *max_wus_to_send* = 1, *max_wus_in_progress* = 2 - ограничение числа выдаваемых задач

5.4 Подбор оптимальных параметров для алгоритма

Веса кастомного алгоритма ($w_{size} = 2$, $w_{dl} = 5$, $w_{rt} = 0.55$, $T_{target} = 30$ s) подбирались итеративно по серии прогонов baseline vs hybrid в нестабильной среде с отказами до получения устойчивого преимущества hybrid по равномерности распределения задач при приемлемой потере в общей продолжительности работы и пропускной способности. Ключевые решения:

- Установка множителя baseline в значение $(10 + 10 \cdot a_j)$ - кастомные слагаемые получают влияние на оценку, но уже посчитанный внутри BOINC score не обнуляется, поскольку цель проекта модифицировать текущую систему, а не написать с нуля. Увеличение коэффициента приводит к слишком высокой корреляции между обеими политиками; излишнее уменьшение - к минимизации вклада в итоговую сумму и как следствие потере смысла. Оптимальным по метрикам оказался коэффициент от 15 до 30 (при фиксированных остальных параметрах), после выявления которого была добавлена привязка к доступности - решение для более качественного учета churn
- $T_{target} = 30$ s - получилось сопоставимо с длительностью medium-задач. Увеличение или уменьшение приводило к нарушению равномерности распределения задач между хостами и заметно ухудшало Jain

- $w_{\text{fit}} = 0.55$ - баланс между штрафом тяжелых задач на слабых хостах и допустимым makespan

Увеличение w_{size} или w_{fit} на 30-50% повышает Jain, но увеличивает makespan hybrid на 5-15%; уменьшение возвращает поведение к baseline. Исходя из изложенного выше, были зафиксированы параметры для алгоритма, после чего тот стал готов к полноценному тестированию.

5.5 Сопоставление подходов в контексте BOINC

Таблица 5.2: Классические методы и их реализация на pull-модели BOINC

Метод	Способ анализа	Ограничение в BOINC
Round Robin	Глобальная очередь	Только local RPC
Least Loaded	Мониторинг всех узлов	Только состояние текущего хоста
LPT / SJF	Централизованный порядок	Ранжирование только внутри feeder
Random	Равновероятный выбор	Детерминированный хэшем
Hybrid	Составная эвристика	Решение для одного клиента

5.6 Ограничения разработанного подхода

- Локальность решений - планировщик не видит глобального распределения задач между однородными хостами: они имеют схожий baseline score, поэтому возможна неравномерность распределения похожих задач между такими хостами, что на самом деле не так и важно при сохранении схожей нагрузки на однородных узлах
- Зависимость от оценок - $\hat{t}_{ij}$ строится на rsc_flops_est , которое может расходиться с реальным CPU-time
- Компромисс Jain vs makespan - усиление $host_fit$ и штрафов за большие задачи на слабых узлах улучшает равномерность и распределение по типам устройств, но увеличивает суммарное время работы относительно других политик, которые "сливают" мелкие задачи на кластеры и минимизируют продолжительность работы
- Нет миграции при churn - если хост перестал быть доступен с задачей in-progress, та ждёт таймаута и репликации; кастомный алгоритм лишь снижает вероятность отправки тяжёлых задач на нестабильные хосты до их назначения

6 Результаты тестирования и сравнение

Проведено шесть полных прогонов по всем политикам. Ниже - метрики по каждому сценарию и сводное сравнение. В таблицах используются следующие столбцы:

- Policy - текущая политика
- Throughput - пропускная способность (задач в секунду)
- Response - среднее время ожидания взятия задачи в работу (в секундах)
- Response p95 - 95-й перцентиль response
- Makespan - общее время выполнения набора задач (в секундах)
- Load CV - ковариация нагрузки на узлы
- Jain - равномерность распределения задач

Ниже эксперименты и короткие выводы по ним:

6.1 Стабильная среда

Таблица 6.1: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.117	124.2	187.5	252	0.368	0.725
round_robin	0.116	117.7	197.5	266	0.449	0.812
weighted_least_loaded	0.109	117.6	206.5	274	0.711	0.761
sjf	0.109	119.2	214.0	272	0.453	0.818
random	0.111	156.9	220.0	269	0.350	0.628
lpt	0.095	144.8	190.3	324	0.451	0.688
hybrid	0.096	140.4	217.8	338	0.758	0.923

Как видим, кастомный метод проиграл планировщику по умолчанию (да и остальным политикам) по всем временным метрикам: продолжительность работы (338 секунд, на 30% больше, чем у baseline), пропускная способность (в среднем на 10-15% хуже остальных), время простоя задач (в p95 хуже него только random). Классические методы Round Robin и Weighted Least Loaded приближены по качеству к baseline, однако тот показывает лучшую ковариацию нагрузки (0.368).

Эвристики также заметно проигрывают по скорости работы планировщику ”из коробки”. Единственная метрика, по которой самописный планировщик опережает другие политики - это равномерность распределения задач по хостам - 0.923, что на 15-30% лучше остальных алгоритмов балансировки. Ниже представлена статистика по хостам:

Таблица 6.2: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	12	10	5	3	4	2	36
round_robin	10	10	5	4	3	4	36
weighted_least_loaded	9	12	5	4	3	3	36
sjf	9	10	7	3	3	4	36
random	14	10	6	2	2	2	36
lpt	11	12	5	2	4	2	36
hybrid	8	8	6	3	6	5	36

По данной таблице видно преимущество разработанного алгоритма - за счет своей архитектуры он не позволяет кластерам забирать бóльшую часть задач для исполнения, а равномерно распределяет их по всем хостам (суммарно на оба кластера приходится 44% задач против 61% у планировщика по умолчанию). Более детально качество такого распределения видно на таблице ниже со статистикой разбора задач каждым хостом по типам:

Таблица 6.3: Число завершённых задач по типам и хостам

	Host						Host				
		short	medium	long	Σ			short	medium	long	Σ
baseline	cluster-01	7	4	1	12	hybrid	cluster-01	3	4	1	8
	cluster-02	5	2	3	10		cluster-02	1	2	5	8
	desktop-01	2	2	1	5		desktop-01	2	4	0	6
	low-power-01	1	2	0	3		low-power-01	1	2	0	3
	phone-01	3	1	0	4		phone-01	6	0	0	6
	phone-02	0	1	1	2		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

Планировщик по умолчанию отдает почти все short-задачи (66%) мощным кластерам, так что слабые устройства вынуждены разбирать средние и тяжелые задачи, тогда как в случае с ка-стомным алгоритмом кластеры забирают все long-задачи и половину medium, оставляя легкие маломощным узлам.

6.2 Нестабильная среда: $f_{\text{off}} \approx 32\%$

Churn: online 25 s, offline 12 s.

Таблица 6.4: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.121	119.4	198.8	243	0.329	0.806
round_robin	0.112	120.2	209.3	277	0.465	0.857
weighted_least_loaded	0.103	133.9	229.0	295	0.459	0.893
sjf	0.110	129.4	223.5	290	0.587	0.851
random	0.085	161.5	213.3	363	0.399	0.610
lpt	0.122	141.4	196.8	237	0.344	0.617
hybrid	0.090	146.8	229.5	344	0.654	0.871

Таблица 6.5: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	11	9	5	4	4	3	36
round_robin	9	9	7	4	4	3	36
weighted_least_loaded	8	9	7	4	4	4	36
sjf	9	10	5	4	4	4	36
lpt	12	13	5	2	2	2	36
hybrid	8	10	5	3	5	5	36
random	9	15	6	2	2	2	36

Таблица 6.6: Число завершённых задач по типам и хостам

baseline	Host	short	medium	long	Σ	hybrid	Host	short	medium	long	Σ
	cluster-01	7	4	1	11		cluster-01	3	0	5	8
	cluster-02	4	2	3	9		cluster-02	3	6	1	10
	desktop-01	2	2	1	5		desktop-01	1	4	0	5
	low-power-01	3	1	0	4		low-power-01	1	2	0	3
	phone-01	2	1	1	4		phone-01	5	0	0	5
	phone-02	0	3	0	3		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

Видно, что в нестабильной среде разработанный алгоритм показывает схожие со стабильной средой результаты: равномерность распределения задач (0.871) так же лучше, чем у любой другой политики; кластеры забирают на себя все сложные задачи, оставляя легкие для более слабых узлов. Он все еще проигрывает по времени (makespan 344 секунды, что значительно больше, чем у остальных), но потери в ключевых метриках составляют около 10-15%, что является по своей сути компромиссом для лучшего распределения задач.

6.3 Нестабильная среда: $f_{\text{off}} = 20\%$

Churn: online 40 s, offline 10 s.

Таблица 6.7: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
baseline	0.119	134.9	186.3	255	0.477	0.844
round_robin	0.102	127.9	217.5	285	0.591	0.850
weighted_least_loaded	0.111	127.0	221.0	276	0.519	0.857
sjf	0.103	128.8	220.5	294	0.608	0.857
random	0.114	145.3	216.3	262	0.450	0.610
lpt	0.113	150.3	213.8	264	0.320	0.610
hybrid	0.097	142.7	214.5	335	0.758	0.871

Таблица 6.8: Число завершённых задач по хостам

Policy	cluster-01	cluster-02	desktop-01	low-power-01	phone-01	phone-02	Σ
baseline	10	9	5	3	5	4	36
round_robin	9	10	5	4	4	4	36
weighted_least_loaded	9	9	7	4	4	3	36
sjf	9	9	7	4	3	4	36
random	14	11	5	2	2	2	36
lpt	14	11	5	2	2	2	36
hybrid	8	10	5	3	5	5	36

Таблица 6.9: Число завершённых задач по типам и хостам

baseline	Host	short	medium	long	Σ	hybrid	Host	short	medium	long	Σ
	cluster-01	6	2	2	10		cluster-01	0	3	5	8
	cluster-02	4	2	3	9		cluster-02	5	4	1	10
	desktop-01	1	3	1	5		desktop-01	2	3	0	5
	low-power-01	0	3	0	3		low-power-01	1	2	0	3
	phone-01	4	1	0	5		phone-01	5	0	0	5
	phone-02	3	1	0	4		phone-02	5	0	0	5
	Σ	18	12	6	36		Σ	18	12	6	36

В этом эксперименте baseline продолжает лидировать по makespan (255 секунд) и пропускной способности (0.119), а кастомный алгоритм сохраняет лучшее качество распределения (0.871) и снова проигрывает по makespan на 30%. По сравнению с прошлым экспериментом самописный алгоритм теряет меньше по пропускной способности (-19% против -26% ранее).

Отметим, что по таблице может сложиться мнение о некачественном распределении long между реер-кластерами (например, 5:1). Это следствие локальности pull-модели: планировщик не видит глобальную загрузку cluster-01/02. При этом phone/low-power не получают long, а Jain остаётся лучшим среди политик.

6.4 Нестабильная среда: $f_{\text{off}} = 10\%$

Churn: online 44 s, offline 5 s.

Таблица 6.10: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
hybrid	0.126	135.2	195.8	226	0.597	0.923
round_robin	0.121	121.1	190.3	252	0.561	0.878
random	0.121	144.9	197.8	254	0.432	0.639
lpt	0.115	152.1	210.0	258	0.318	0.643
sjf	0.118	115.6	215.3	265	0.468	0.837
weighted_least_loaded	0.113	119.3	210.0	270	0.622	0.761
baseline	0.094	137.6	200.3	350	0.679	0.831

В этом сценарии hybrid впервые лидирует по makespan (226 s) и throughput (0.126), сохраняя лучший Jain (0.923). Baseline проседает (makespan 350 s) - при редких, но частых RPC churn availability-aware ranking перераспределяет long с нестабильных узлов эффективнее, чем чистый deadline-score.

6.5 Нестабильная среда: $f_{\text{off}} = 50\%$

Churn: online 30 s, offline 30 s.

Таблица 6.11: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
random	0.109	171.5	222.5	289	0.474	0.639
baseline	0.104	153.1	221.0	287	0.494	0.806
lpt	0.106	165.0	233.0	294	0.377	0.587
round_robin	0.097	140.7	226.8	320	0.344	0.931
sjf	0.102	145.0	230.3	304	0.279	0.871
hybrid	0.093	157.5	234.3	339	0.622	0.900
weighted_least_loaded	0.085	145.9	255.5	370	0.590	0.878

При 50% offline round_robin достигает Jain=0.931 (лучший в сценарии), baseline - лучший makespan (287 s). Hybrid сохраняет Jain $\geq$ 0.90, но проигрывает по времени - при длительных offline size-aware политика консервативно отдаёт short на edge-узлы, увеличивая makespan.

6.6 Нестабильная среда: $f_{\text{off}} \approx 68\%$

Churn: online 12 s, offline 25 s.

Таблица 6.12: Метрики

Policy	Throughput	Response	p95	Makespan	Load CV	Jain
lpt	0.115	186.6	240.0	260	0.346	0.610
round_robin	0.100	148.6	269.8	305	0.456	0.885
baseline	0.100	192.2	273.3	306	0.498	0.812
sjf	0.104	149.3	249.3	302	0.517	0.893
weighted_least_loaded	0.099	151.9	265.5	317	0.485	0.931
random	0.088	179.3	250.0	360	0.626	0.735
hybrid	0.084	179.9	289.0	377	0.624	0.915

При экстремальном churn лидирует lpt по makespan (260 s); hybrid сохраняет Jain=0.915, но makespan 377 s - availability-aware штрафы и size-matching не компенсируют частые длинные простои. Weighted least loaded достигает Jain=0.931, baseline - средний баланс.

6.7 Сводное сравнение сценариев

Таблица 6.13: Лидеры по makespan и Jain в каждом прогоне

f_{off}	Best makespan	Makespan (s)	Best Jain	Jain	Hybrid: Jain / Makespan
Stable FULL	baseline	252	hybrid	0.923	0.923 / 338
32	lpt	237	wleast	0.893	0.871 / 344
20	baseline	255	hybrid	0.871	0.871 / 335
10	hybrid	226	hybrid	0.923	0.923 / 226
50	baseline	287	round_robin	0.931	0.900 / 339
68	lpt	260	wleast	0.931	0.915 / 377

Таблица 6.14: Throughput (tasks/s) hybrid и baseline по сценариям

Policy	Stable	32	20	10	50	68
baseline	0.117	0.121	0.119	0.094	0.104	0.100
hybrid	0.096	0.090	0.097	0.126	0.093	0.084

6.8 Интерпретация результатов

- Baseline оптимален по makespan/throughput в stable и умеренном churn (40/10, 30/30), минимизируя общее время через концентрацию short на кластерах
- Разработанный алгоритм стабильно лидирует по Jain (0.87-0.92) во всех сценариях; единственный случай победы по makespan - средняя доступность в 10%, где короткие offline-окна и availability-aware ranking совпадают с целями алгоритма
- LPT выигрывает по makespan, но Jain низкий (0.61-0.62) - long концентрируются на cluster
- Round Robin / Weighted Least Loaded - компромисс: близки к baseline по времени, Jain выше из-за особенностей алгоритмы (распределять задачи по хостам примерно поровну)
- Random был добавлен скорее как второй baseline, чтобы разработанный алгоритм не оказался вдруг хуже случайного, и он очень нестабилен от запуска к запуску, что было ожидаемо
- Наблюдается устойчивый trade-off: +15-30% у времени работы hybrid относительно baseline при выигрыше в 15-27% Jain и корректном распределении short/long задач по классам устройств

7 Заключение

Реализован прототип локальной эвристики выдачи задач для гетерогенной BOINC-среды и контейнерный стенд для его сравнения со штатным планировщиком. Модификация сохраняет pull-модель и использует штатные механизмы проверки допустимости и выдачи задания. Основным результатом работы - воспроизводимое описание поведения выбранной конфигурации и выявленный компромисс между распределением числа задач и временем завершения набора.

7.1 Подтверждённые результаты

- 1 Реализованы семь политик планирования задач: baseline, random, lpt, sjf, weighted_least_loaded, round_robin, hybrid
- 2 Построен Docker-стенд из 6 гетерогенных хостов с контролируемы отказами; проведено полноценное тестирование всех политик в разных условиях со сбором метрик
- 3 Разработанный алгоритм стабильно обеспечивает наилучшую равномерность (0.87-0.92) и size-aware распределение: большие задачи на мощные узлы, а мелкие на более слабые - чего не гарантировал никакой другой алгоритм
- 4 В сценарии с кратковременными отказами (доступность на уровне 90%) кастомный алгоритм одновременно лидирует по общей продолжительности работы (226 s), пропускной способности (0.126) и равномерности распределения (0.923) - доказывая, что метод применим не только в нестабильной среде, но и в приближенной к стабильной, где у штатного планировщика он выигрывает уже по большинству метрик

7.2 Ограничения и компромиссы

- В большинстве сред политики baseline/LPT быстрее на 10-30% по общему времени разбора задач в сравнении с разработанным алгоритмом - сознательно жертвуем скоростью ради равномерности и честности распределения
- Pull-модель BOINC не даёт глобального баланса между однородными кластерами (при всех прогонах распределение наиболее тяжелых задач 1 к 5) - баланс достигается комплексно за счет грамотного распределения всех типов задач
- Оценки rsc_frops_est и локальный учет активных задач на хосте ограничивают точность алгоритма в реальной среде

В общем и целом цель работы - разработка системы выдачи заданий для гибридной BOINC-среды - достигнута: алгоритм разработан и реализован, протестирован в различных сценариях. Также были зафиксированы преимущества, недостатки и компромиссы, на которые пришлось пойти для улучшения метрик.

Список литературы

- [1] Ali M. Alakeel. «A Guide to Dynamic Load Balancing in Distributed Computer Systems». В: *International Journal of Computer Science and Network Security* 1.2 (2010).
- [2] David P. Anderson. «BOINC: A Platform for Volunteer Computing». В: *Journal of Grid Computing* 18 (2020), с. 99—122. DOI: [10 . 1007 / s10723 - 019 - 09497 - 9](https://doi.org/10.1007/s10723-019-09497-9). URL: [https : / / boinc . berkeley.edu/boinc_a_platform_for_volunteer_computing.pdf](https://boinc.berkeley.edu/boinc_a_platform_for_volunteer_computing.pdf).
- [3] Tony Bourke. *Server Load Balancing*. O'Reilly & Associates, 2001.
- [4] I. I. Kurochkin и А. А. Dolgov. «Load balancing in grid systems from personal devices on the BOINC platform». В: *XIV All-russian multi-conference on management issues of the MCPU-2021*. 2021.
- [5] Mazedur Rahman, Samira Iqbal и Jerry Gao. «Load Balancer as a Service in Cloud Computing». В: *IEEE 8th International Symposium on Service Oriented System Engineering (SOSE)*. 2014.
- [6] Lun Wang. «Low-Latency, High-Throughput Load Balancing Algorithms». В: *Journal of Computer Technology and Applied Mathematics* 10.6 (2024), с. 153—160.
- [7] М. О. Манзюк, О. С. Заикин и М. А. Посыпкин. «CluBORun: программный комплекс для использования свободных ресурсов вычислительных кластеров в BOINC-расчетах». В: *Информационные технологии и вычислительные системы* 4 (2014), с. 3—11. URL: [https : / / www.isa.ru/jitcs/images/documents/2014-04/3_11.pdf](https://www.isa.ru/jitcs/images/documents/2014-04/3_11.pdf).